03. CRUD Operations, HTTP Status Codes & JSON

1. The 4 Primary Actions: What is CRUD?

Almost every feature in modern software revolves around four core actions known as **CRUD**:

- **C**reate: Adding brand-new data to the system.
- **R**ead: Fetching and viewing existing data.
- **U**pdate: Modifying existing data.
- **D**elete: Removing data permanently.

In web development, each CRUD operation maps directly to an **HTTP Method (Verb)**:

CRUD Operation	HTTP Method	What It Tells the Server	Example Endpoint	Real-World Example
Create	`POST`	"Save this new record."	`POST /students`	Enrolling a new student.
Read	`GET`	"Give me data. Do not alter anything."	`GET /students`	Viewing the list of enrolled students.
Update	`PUT`	"Replace/update this existing record."	`PUT /students/1`	Updating a student's contact details.
Delete	`DELETE`	"Remove this record permanently."	`DELETE /students/1`	Removing an unenrolled student record.

Safe vs. State-Modifying Methods:

- `GET` **is Safe (Read-Only):** A user can refresh a `GET` request 100 times, and the data on the server will never change.
- `POST`, `PUT`, **and** `DELETE` **modify state:** Every execution creates, modifies, or deletes records on the backend.

---

2. HTTP Status Codes: The Server's Response Language

When the server sends back an answer, it includes a **3-digit Status Code** so the frontend knows what happened without guessing.

You can remember all status codes using the **first-digit rule**:

```
2xx ===> SUCCESS ("Everything worked as requested!") 3xx ===> REDIRECTION ("The resource has moved to a new address.") 4xx ===> CLIENT ERROR ("The client made a mistake in the request.") 5xx ===> SERVER ERROR ("The server crashed due to an internal bug.")
```

The Most Common Status Codes:

Code	Name	Category	When It Is Used
`200`	OK	Success	Standard success response for `GET`, `PUT`, or `DELETE`.
`201`	Created	Success	A new record was successfully created via `POST`.
`400`	Bad Request	Client Error	The client sent missing or malformed data syntax.
`404`	Not Found	Client Error	The requested endpoint URL or record ID does not exist.
`422`	Unprocessable Entity	Client Error	Validation error in FastAPI (e.g., sent text where an integer was expected).
`500`	Internal Server Error	Server Error	The Python backend code crashed or threw an unhandled exception.

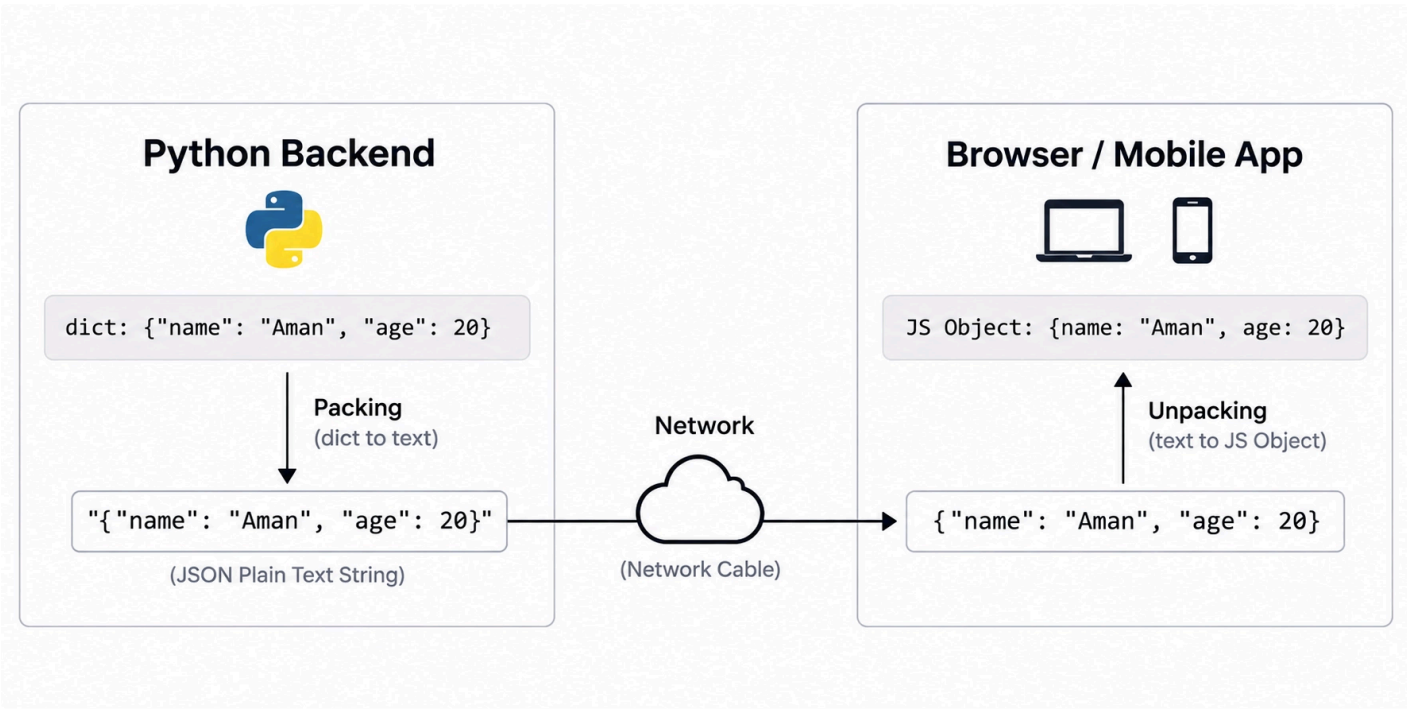
---

3. Demystifying JSON (The Universal Data Format)

1. The Core Concept: Memory vs. Text

- **A Python Dictionary** is a live data structure that exists only inside your computer's memory (RAM) while Python is running. You cannot send raw computer memory across a network wire.
- **JSON (JavaScript Object Notation)** is simply a **plain-text format** (a string of characters). Because it is plain text, every programming language in the world (Python, JavaScript, Dart, Swift, Java, C++) can read and write it.

Even though it has "JavaScript" in its name, **JSON does not belong to JavaScript**. It is the universal standard for sending data over the web.



---

2. The Concepts of Packing & Unpacking

Whenever applications communicate over the internet, data goes through two simple steps:

1. **Packing (Serialization):** Converting structured data from your program's memory into a plain JSON text string so it can travel across the network wire.
2. **Unpacking (Deserialization):** Taking an incoming JSON text string from the network and converting it back into structured data that your program can read and work with.

FastAPI handles both automatically:

- When an app sends data to your backend, FastAPI unpacks the JSON text into Python automatically.
- When your Python function returns data, FastAPI packs it into a clean JSON text response for the client automatically.

---

3. Python Dictionary vs. JSON:

Feature	Python Dictionary	JSON (Plain Text)
What is it?	Live data structure in Python RAM	Plain text string
Who understands it?	Only Python	Every programming language in the world
Quotes Rule	Single `'name'` or double `"name"` quotes	**Must always be double quotes** `"name"`
Keys	Can be strings, numbers, tuples, booleans	**Must always be strings**
Booleans	`True` / `False` (Capitalized)	`true` / `false` (Lowercase)
Null / Empty value	`None`	`null`